

LUT core-module ablation: three generations of lookup math

Comparison table

2026-09-13

Protocol

Architecture: LUT transformer, $E = 384$, 6 layers, 6 attention heads. FFN = **CompressionMultiHeadLUT** with $H = 4$ heads $\times$ $tph = 128$ tables ($P = H \cdot tph = 512$ tables per token), $d_{in} = d_{out} = 48$, $n = 8$ anchor pairs per table ($K = 2^n = 256$ cells); **compress** $384 \rightarrow 192$, **decompress** $192 \rightarrow 384$. **Training:** 16,000 steps, batch size 48 (24576 tokens), untied embeddings, LUT tables without weight decay. **Eval:** **evaluate_bpb_fixed**, 48 rows $\times$ 100 steps, first 12 rows skipped.

Counting convention

- Core LUT only, per token, one layer. The **compress/decompress** projections are identical in every row and excluded (147,456 MACs forward).
- **FLOP** = one scalar float operation ($+$, $-$, $\times$, $\div$, a comparison, $|\cdot|$, $\exp$, $\log \sigma$, σ). **MAC** = a multiply accumulated into a sum; 1 MAC = 2 FLOPs, included in the FLOP column. Memory reads and integer work are not counted.
- **Forward** = inference. **Backward** = all additional training arithmetic, including per-table gradients of per-layer scalars and training-only regularisers (amortised over the 24,576 tokens of a step).

Table A — forward function and gradient function

Per variant, (a) is the function computed on the forward pass and (b) the function whose gradient training uses: the weights always receive the gradient of (a), while the input, the temperatures and the learned per-layer scalars receive the gradient of (b) with the table weights held constant.

Variant	(a) Forward: the function computed	(b) Backward: the function whose gradient is used
GEN 1 — Spiking Manifesto basic model		
1.1 Hard forward, two-alternatives backward	$y = \sum_t W_t[c_t]$, summed over tables t : each of table t 's n anchor pairs gives a signed margin $u_j = z[a_j] - z[b_j]$; the n signs $[u_j > 0]$ are the bits of the binary index $c_t \in \{0, \dots, 2^n - 1\}$ of the row read from $W_t \in \mathbb{R}^{K \times d}$, $K = 2^n$.	The 2-cell blend $\sum_t [(1 - U_t)W_t[c_t] + U_tW_t[c'_t]]$, $U_t = U(u_{j^*})$, $U(u) = 0.5/(1 + u)$, with the least-confident pair $j^* = \arg \min_j u_j $ and c' (c with bit j^* flipped) held fixed. The pairing with (a) is exact as a stop-gradient composite; the blend is C^1 across a bit flip and jumps only where j^* switches.
1.2 Soft forward, two-alternatives backward	(a) = (b): the blend of 1.1(b), applied at eval too.	
GEN 2 — FastMHL math		
2.1 Hard forward, all-alternatives backward (all K cells)	$y = \sum_t W_t[c_t]$, as 1.1(a).	With $b_j(\ell) = +1$ if the bit of ℓ for pair j is 1, else -1 , and the squashed margin $\hat{m}_j = u_j/(T_s + u_j) \in (-1, 1)$: $s_\ell = \sum_j b_j(\ell) \hat{m}_j$ scores how well row ℓ 's bit pattern agrees with the actual signs of the margins (small margins count little either way). All K rows of each table are read, softmax-weighted within that table: $\sum_t \sum_{\ell < K} \text{softmax}_\ell(s_\ell/T_{sel}) W_t[\ell]$. T_s, T_{sel} : per-layer scalars, learned in <code>exp_n_0121</code> . Globally C^1 in the input; never uses the address, whose row is the one with the largest weight.
2.2 Hybrid_smooth two-alternatives forward, all-alternatives backward	$y = \sum_t [(1 - w_t)W_t[c_t] + w_tW_t[c'_t]]$, $w_t = \sigma(-2\rho_{j^*}/T_{sel})$, $\rho_j = u_j /(T_s + u_j)$, σ the logistic sigmoid; applied at eval too.	The same softmax-weighted read of all K rows as 2.1(b).
2.3 Hard forward, 2-alternatives backward	$y = \sum_t W_t[c_t]$	TBD — not implemented; open decision. <code>FastMultiHeadLut</code> raises <code>ValueError</code> for <code>backward_topk>0</code> unless <code>forward_mode='hybrid_smooth'</code> .
2.4 Hybrid_smooth two-alternatives forward, 2-alternatives backward	(a) = (b): the 2-cell blend of 2.2(a) (<code>backward_topk=1</code> : it counts cells beyond the addressed one, so 2 alternatives is 1, not 2).	
GEN 3 — LightMHL math (LookupFFN-inspired): learned margin score		
3.1 Learned margin score, per-layer (g, β, γ)	(a) = (b): $y = \sum_t s_t W_t[c_t]$ with the address c_t held fixed; $s_t = e^g S_t P_t^\gamma$, $S_t = \sum_j m_j$, $P_t = \prod_j \sigma(\beta m_j)$, $m_j = u_j $, computed as $S_t \exp(g + \gamma \sum_j \log \sigma(\beta m_j))$; one ($g, \log \beta, \log \gamma$) per layer, initialised at $(0, 2, 1)$. Discontinuous at cell boundaries, since $\sigma(0) > 0$ (trained jump: median 2.19% of $\ y\ $).	
3.2 The Gen-3 form: learned margin score, per-layer (β, γ)	(a) = (b): 3.1 with g fixed at 0, $s_t = S_t P_t^\gamma$ (jump 2.22%).	
3.3 = 3.2 + Hamming-1 cell TV, $\lambda = 10$	(a) = (b): 3.2 (jump 1.78%); training adds $\lambda \cdot \frac{1}{6} \sum_{\text{layers}} \frac{1}{P_n 2^{n-1}} \sum_t \sum_{(c, c')} \ W_t[c] - W_t[c']\ ^2$, over cell pairs differing in one bit, to the loss.	

Table B — cost and results

Per token, one layer, core LUT only; $P = 512$, $n = 8$, $K = 256$, $d = 48$.

Variant	Forward FLOPs	Backward FLOPs	Forward MACs	Backward MACs	Validation bpb	Implementation (class used)
1.1	$P(2n + d) = \mathbf{32,768}$	$P(5d + 2n + 9) = \mathbf{135,680}$	0	$P \cdot 2d = \mathbf{49,152}$	TBD — not yet implemented	MultiHeadLut(smooth_mode=False, n_alternatives=1, uncertainty_mode=INVERSE_L1); not wired into CompressionMultiHeadLUT.
1.2	$P(4d + 4n + 4) = \mathbf{116,736}$	$P(8d + 9) = \mathbf{201,216}$	$P \cdot 2d = \mathbf{49,152}$	$P \cdot 4d = \mathbf{98,304}$	TBD — not yet implemented	MultiHeadLut(smooth_mode=True, n_alternatives=1); not wired in.
2.1	$P(2n + d) = \mathbf{32,768}$	$P(2Kd + 4nK + 9K + 11n + d) = \mathbf{18,026,496^e}$	0	$P(Kd + 2nK) = \mathbf{8,388,608}$	1.180622 (exp_n_0121) ^f	FastMultiHeadLut(forward_mode='hard', backward_topk=0).
2.2	$P(2kd + 4n + 5) = \mathbf{117,248}$, $k = 2$ cells read	$P(2Kd + 4nK + 9K + 11n + 2kd) = \mathbf{18,100,224^e}$	$Pkd = \mathbf{49,152}$	$P(Kd + 2nK + kd) = \mathbf{8,437,760}$	TBD — not yet run	FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=0).
2.3	$P(2n + d) = 32,768$	n/a	0	n/a	TBD — not implemented; open decision	Would be FastMultiHeadLut(forward_mode='hard', backward_topk=1), which the code rejects.
2.4	$P(2kd + 4n + 5) = \mathbf{117,248}$	$P(4kd + 9n + 15) = \mathbf{241,152}$	$Pkd = \mathbf{49,152}$	$P2kd = \mathbf{98,304}$	TBD — not yet run	FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=1).
3.1	$P(2kd + 7n + 2) = \mathbf{78,848}$, $k = 1^g$	$P(4kd + 10n + 10) = \mathbf{144,384^g}$	$Pkd = \mathbf{24,576}$	$P2kd = \mathbf{49,152}$	1.169344 (exp_g_0247)	LightMultiHeadLUT(confidence_form='learned_margin', learned_margin_init=(0, 2, 1)).
3.2	$P(2kd + 7n + 2) = \mathbf{78,848}$	$P(4kd + 10n + 8) = \mathbf{143,360^g}$	24,576	49,152	1.169328 (exp_g_0248)	As 3.1 with learned_margin_freeze_g=True.
3.3	78,848 (as 3.2)	149,504 (3.2 + TV) ^h	24,576	51,200^h	1.163698 (exp_g_0249)	As 3.2, plus (10 * model.lut_tv_penalty()).backward() per step (lut_cell_smoothness: 10.0).

Notes to Table B.

^e Dominated by $2Kd$: every table's K rows are read; several [tokens, 512, 256] fp32 buffers of about 12.9 GiB at 24,576 tokens.

^f Re-score of the saved run under the corrected eval (0 missing keys); its in-run batch-coupled number was 1.191455.

^g Per table, forward: n comparisons, n differences, n absolute values, n products βm_j , $n \log \sigma$, the two sums, $\times \gamma$, $+g$, exp and the final product $(7n + 2)$, plus the scaled row $(2kd)$. Backward: the row terms $(4kd)$, the input term $(8n + 3)$ and the per-table accumulation of the parameter gradients $(\partial g_2, \partial \log \gamma_3, \partial \log \beta_2n + 2)$; 3.2 drops ∂g . The parameter gradients are scalar multiplies, so they add no MACs.

^h Training only, once per step: $Pn2^{n-1} = 524,288$ cell pairs per layer, $3d$ FLOPs and d MACs per pair in each direction (150,994,944 FLOPs, 50,331,648 MACs per layer per step), i.e. +6,144 FLOPs and +2,048 MACs per token over 24,576 tokens. Inference cost is 3.2's.

The Gen-3 form

- **Learned margin score with per-layer trainable β and γ (3.2). Drop g :** freezing it leaves bpb (-0.000016 vs 3.1) and the γ profile ($\max |\Delta\gamma|$ 0.016) unchanged. Trained γ 1.04–1.60, highest mid-depth; β 1.76–2.17 and nearly irrelevant to the score's shape. Both were still drifting at 16K.
- **Cell TV (3.3): -0.0056 vs 3.2, a mild regularisation benefit. The mechanism is mostly table shrinkage, not smoothing:** table L2 norms fell to $0.22\text{--}0.35\times$ of 3.2's, while the scale-free ratio $\|W[c] - W[c']\|^2 / \|W[c]\|^2$ (2.0 for independent random cells) moved only from 1.74–1.95 to 1.34–1.57 across layers.

Where the code differs from the original brief

- 1.1’s input gradient is a surrogate (the derivative of 1.2’s blend); its weights update only the addressed row.
- The status report’s $u(\delta) = 0.5/(T + |\delta|)$ has a temperature; the code’s $0.5/(1 + |u|)$ has none (`l_projection.py:96`, `lutorch.cu:797`).
- Gen 2’s hybrid_smooth weight $\sigma(-2\rho_{j^*}/T_{sel})$ is a temperature-controlled soft-sign, not Gen 1’s $U(u)$.

Remaining work

- **Run 2.2 and 2.4:** fork `exp_n_0121` verbatim, changing only `lut_forward_mode` / `lut_backward_topk`. No matching run exists (earlier hybrid_smooth runs are $n = 6$, tied, old eval; `exp_n_0188` is $n = 7$ with a gate on).
- **Implement 1.1, 1.2:** wire Gen 1 into `CompressionMultiHeadLUT`, which accepts only `lut_impl` $\in \{\text{fast, light, bh4}\}$ (`compression_mh1.py:190`), as `MultiHeadLut` with `anchor_candidates` `[128, 4, 48]`. **Open decision, 2.3:** patch `FastMultiHeadLut` (`fast_multi_head_lut.py:1417`) or drop the row. A hard-forward Gen-3 variant has not been designed.
- **Seeds:** zero replicates at this geometry; one extra seed of `exp_g_0248` (≈ 0.95 h) would give a real LUT seed spread. Reference: vanilla dense FFN 1.1651 / 1.1618 at 16K (two seeds), 1.1154 at 48K.